

Solar Power Forecasting API

Input Data Reference Guide

Global Deployment Edition

LSTM-based 24-hour-ahead PV power forecasting
Works for any solar plant, anywhere on Earth

1	User-Provided Data — Plant Registration (any location)
2	Weather API Data — Per Prediction (192 rows × 15 min)
3	Additional Inputs Summary & panel_temperature note

PAGE 1 — User-Provided Data (Plant Registration)

Entered **once per plant** by the user or plant operator. Does not change between predictions. This data is stored in your system and sent to the API with every request.

Section 1A — Plant Identity

For display and routing only — not sent to the model.

Field	Type	Example	Notes
Plant Name	text	Solar Farm Kenya 1	Display only — not sent to model
Plant ID	integer	52	Assigned by your system for new plants

Section 1B — Technical Specifications (META_COLS — 7 fields, exact order)

These 7 fields map **exactly** to META_COLS in the trained model. The order is fixed. They are found on the plant commissioning document and panel datasheet.

#	Field	Unit	Example	Source
1	Nominal Power	MW	50.0	Plant commissioning document
2	Number of Panels	count	100,000	Plant commissioning document
3	Panel Efficiency	%	21.5	Panel datasheet
4	Temperature Coefficient	%/°C	-0.35	Panel datasheet (negative)
5	Bifaciality Coefficient	0.0–1.0	0.70	Panel datasheet (0 = not bifacial)
6	Structure Type	TRACKER / FIXED TRACKER		Plant design document
7	Country / Region	text	Kenya	Plant location (replaces Brazilian state)

Note: The original model was trained on Brazilian plants using a `brazil_federative_unit` state code. For global deployment, replace `state_code` with a country/region integer derived from your own region map. The model's meta branch will adapt to any consistent integer encoding — what matters is consistency between training and inference.

Section 1C — Location Encoding for Global Deployment

The original model used a `state_code` integer derived from Brazil's federal states. For global deployment you must define your own location map and retrain or fine-tune the metadata branch. Two recommended approaches:

Approach	How it works	Best for
Country code	Map each country to an integer (0–195). Use ISO 3166-1 numeric codes.	Small number of countries
Climate zone	Map Köppen climate zone (e.g. BWh=0, Aw=1) to integer. Can use solar irradiance patterns.	More solar irradiance patterns better than geography.
Latitude band	Divide latitude into 5° bands (e.g. -90 to 90 → 36 integers). Directly encodes solar angle.	Directly encodes solar angle.

Whichever approach you choose, apply it consistently: fit the `meta_scaler` on the new integer column during retraining, and send the same integer at inference time.

Section 1D — What the API Sends (JSON example)

```
{ "metadata": { "nominal_power_mw": 50.0, "number_of_panels": 100000,
"panel_efficiency_percentage": 21.5, "panel_temperature_coefficient": -0.35,
"panel_bifaciality_coefficient": 0.70, "is_tracker": 1, "location_code": 404 },
"weather_sequence": [ ... 192 rows ... ] }
```

Section 1E — Validation Rules (enforce on frontend)

Field	Rule
Nominal Power	must be > 0
Number of Panels	must be > 0, integer
Panel Efficiency	must be between 5.0 and 30.0
Temperature Coefficient	must be between -1.0 and 0.0 (always negative)
Bifaciality Coefficient	must be between 0.0 and 1.0
Structure Type	must be TRACKER or FIXED only
Location Code	must be a valid integer from your location map

PAGE 2 — Weather API Data (Per Prediction Request)

Fetched **automatically** from a weather provider every 15 minutes. The user does not enter this manually. The frontend collects the **last 192 readings** and sends them with every prediction request.

Section 2A — Required Weather Fields (SEQ_FEATURES inputs)

These 7 raw sensor fields are the ONLY weather inputs the frontend sends. All other features (18 total) are computed by the API automatically.

#	Field Name	Unit	Range	Description
1	datetime	ISO 8601	—	Timestamp e.g. 2026-08-01T06:00:00. UTC recommended for global use.
2	poa_irradiance_wm2	W/m²	0 – 1400	Solar radiation hitting the panel surface (tilted). Primary solar signal.
3	ghi_irradiance_wm2	W/m²	0 – 1400	Global Horizontal Irradiance — radiation on a flat surface.
4	ambient_temperature_celsius	°C	-30 – 60	Air temperature at plant location. Extended range for global deployment.
5	panel_temperature_celsius	°C	-20 – 90	Panel surface temperature. Measured sensor or calculated (see Page 3).
6	wind_speed_ms	m/s	0 – 60	Wind speed at hub height. Extended upper bound for global deployment.
7	wind_direction_degrees	degrees	0 – 360	Wind direction. 0=North, 90=East, 180=South, 270=West.

Global range note: ambient_temperature range extended to -30°C – 60°C to cover Arctic installations and equatorial desert sites. wind_speed_ms range extended to 60 m/s for offshore and high-altitude installations.

Section 2B — Sequence Requirements

Requirement	Value
Number of rows required	exactly 192
Time spacing between rows	exactly 15 minutes
Total time window covered	48 hours (last 48h before prediction)
Order	chronological — oldest row first, newest row last
Timezone	Use UTC for global deployment. Apply timezone offset consistently.
Gaps allowed	No — interpolate missing values before sending

Section 2C — Example of One Weather Row

```
{ "datetime": "2026-08-01T06:00:00Z", "poa_irradiance_wm2": 320.5, "ghi_irradiance_wm2": 295.0, "ambient_temperature_celsius": 24.3, "panel_temperature_celsius": 38.7, "wind_speed_ms": 3.2, "wind_direction_degrees": 180.0 }
```

Section 2D — What the API Computes Automatically

Do NOT send these — the API builds them from the 7 raw fields above.

Computed Feature	Built From	Formula
hour_sin, hour_cos	datetime	$\sin/\cos(2\pi \times \text{hour} / 24)$
doy_sin, doy_cos	datetime	$\sin/\cos(2\pi \times \text{day_of_year} / 365)$
month_sin, month_cos	datetime	$\sin/\cos(2\pi \times \text{month} / 12)$
clearness	poa_irradiance_wm2	$\text{poa} / 1000$, clipped [0, 1]

Computed Feature	Built From	Formula
ghi_norm	ghi_irradiance_wm2	ghi / 1000, clipped [0, 1.5]
target_solar_elev	datetime + 24h	Solar declination + hour angle (trigonometry)
target_hour_sin/cos	datetime + 24h	sin/cos of hour angle at target time
StandardScaler transform	all 18 features	Applied using saved training scaler

Section 2E — Global Timezone Handling

For global deployment the datetime field must be handled carefully:

Rule	Detail
Use UTC internally	Store and process all timestamps in UTC. Convert to local time only for display.
Append Z or +00:00	Send UTC times as "2026-08-01T06:00:00Z" or "2026-08-01T06:00:00+00:00"
Solar geometry uses UTC	target_solar_elev is computed correctly from UTC timestamps — no local time conversion needed.
Night mask uses local solar time	NIGHT_START/END (20:00 and 05:00) must be compared against the plant's local solar time, not UTC.

PAGE 3 — Additional Inputs Summary

Section 3A — Existing Plants (already trained)

Input	Type	Required?	Notes
plant_id	integer	Yes	Select from dropdown of trained plant IDs
weather_sequence	automatic	Yes	192 rows from weather API — not entered by user

Section 3B — New Plants (not seen during training)

Input	Type	Required?	Notes
All 7 metadata fields	various	Yes	See Page 1 Section 1B
weather_sequence	automatic	Yes	192 rows — not entered by user

Section 3C — Inputs the User Never Enters

Item	Reason
Scaling / normalisation	Done by API using saved StandardScaler from training
Feature engineering (cyclic, clearness)	Done by API from raw datetime + irradiance
Plant peak power scale	Stored in package for known plants; estimated for new plants
total_active_power_w	Not available at inference time — API uses 0.0
target_solar_elev	Deterministic trigonometry from datetime — no sensor needed

Section 3D — How to Get panel_temperature_celsius Globally

Most global weather APIs (Open-Meteo, Solargis, PVGIS, Tomorrow.io) do not directly provide panel surface temperature. Use one of these methods:

Option A — NOCT thermal model (simplest, no library needed)

```
panel_temp = ambient_temp + (poa_irradiance / 800) * (NOCT - 20) # NOCT is on the panel
datasheet. Typical value: 45°C # Example: ambient=25°C, poa=600 W/m² # panel_temp = 25 +
(600/800) * (45-20) = 43.75°C
```

Option B — pvlib SAPM model (more accurate)

```
import pvlib
panel_temp = pvlib.temperature.sapm_cell( poa_global = poa_irradiance_wm2,
temp_air = ambient_temperature_celsius, wind_speed = wind_speed_ms, a = -3.56, # glass/glass
module constant b = -0.075, deltaT = 3, )
```

Option C — Weather API providers that include it

Provider	Field name	Notes
Solargis	PVTEMP	Direct panel temperature — most accurate
PVGIS	T_module	Available in PVGIS-SARAH2 and ERA5 datasets
Tomorrow.io	panelTemperature	Available on professional plan
Open-Meteo	Not available	Use NOCT model instead
NASA POWER	Not available	Use NOCT model instead

Section 3E — Recommended Global Weather API Stack

API	POA irradiance	GHI	Ambient temp	Panel temp	Wind	Free tier
Open-Meteo	Yes (calc)	Yes	Yes	No (NOCT)	Yes	Yes — unlimited
PVGIS (EU)	Yes	Yes	Yes	Yes	Yes	Yes — free
Solargis	Yes	Yes	Yes	Yes	Yes	No — paid
Tomorrow.io	Yes	Yes	Yes	Paid plan	Yes	Limited free
NASA POWER	Yes	Yes	Yes	No (NOCT)	Yes	Yes — free

Recommendation: Open-Meteo for development (free, global, no key needed). PVGIS for European sites (free, accurate). Solargis for production-grade global forecasting.